

MVP Build Specification: Contextualize.ai

Document Type	MVP Technical Specification
Developer	Solo Dev (You)
Timeline	4 Weeks (Part-time)
Objective	Build a "Walking Skeleton" that captures text, stores vectors, and answers questions.

1. The "Ruthless" Scope (What is IN vs. OUT)

To ensure this is manageable for a solo developer, we must be ruthless about cutting features.

Feature	IN (MVP)	OUT (Save for V2)
Authentication	Google Auth (via Supabase)	Email/Password, SSO, Teams
Ingestion	Browser Extension (Current Tab)	PDF Upload, YouTube, Manual Paste
Content Processing	Text Only (Stripped HTML)	Images, Charts, Tables, OCR
Organization	Auto-generated Title/Summary	Folders, Tags, Edit Metadata
Chat	Single Stream Q&A	Chat History Persistence, Multi-turn reasoning
UI	Dark Mode, Simple Cards	Custom Themes, Drag & Drop

2. Database Schema (Supabase + PostgreSQL)

We need three core tables. Run this SQL in your Supabase SQL Editor to set up the backend instantly.

Enable Vector Extension

```
create extension if not exists vector;
```

Table 1: profiles (Users)

Standard user management linked to Supabase Auth.

```
create table profiles (
  id uuid references auth.users not null primary key,
  email text,
  full_name text,
  avatar_url text,
  created_at timestampz default now()
);
```

Table 2: documents (The Bookmarks)

Stores the metadata and the raw text for reference.

```
create table documents (
  id uuid default gen_random_uuid() primary key,
  user_id uuid references auth.users not null, -- RLS required
  url text not null,
  title text,
  summary text, -- AI generated 1-liner
  content text, -- Stripped text from Readability.js
  created_at timestampz default now(),

  -- Add a match score for search ranking later
  match_score float generated always as (0) stored
);
```

Table 3: document_chunks (The Vectors)

Long articles must be split into chunks (approx 500-1000 tokens) because LLMs have token limits and search is better on smaller segments.

```
create table document_chunks (
  id uuid default gen_random_uuid() primary key,
  document_id uuid references documents(id) on delete cascade,
  content_chunk text, -- The specific paragraph(s)
  embedding vector(1536) -- OpenAI text-embedding-3-small dimension
);
```

Vector Search Function (RPC)

This is the "Magic" function you will call from your Next.js API.

```
create or replace function match_documents (
  query_embedding vector(1536),
  match_threshold float,
  match_count int,
  p_user_id uuid
)
returns table (
  id uuid,
  content_chunk text,
```

```

        similarity float
    )
language sql stable
as $$
    select
        document_chunks.id,
        document_chunks.content_chunk,
        1 - (document_chunks.embedding <=> query_embedding) as similarity
    from document_chunks
    join documents on documents.id = document_chunks.document_id
    where documents.user_id = p_user_id
    and 1 - (document_chunks.embedding <=> query_embedding) > match_threshold
    order by document_chunks.embedding <=> query_embedding
    limit match_count;
$$;

```

3. API Route Architecture (Next.js)

You only need 2 main API endpoints.

A. The Ingestion Endpoint: POST /api/ingest

- **Trigger:** Called by the Chrome Extension when user clicks "Save".
- **Payload:** { url: string, html: string }
- **Workflow:**
 1. **Validate User:** Check Supabase Auth token.
 2. **Clean:** Use @mozilla/readability to convert HTML -> Clean Text.
 3. **Summarize:** Send first 2k chars to OpenAI (gpt-4o-mini) to get a Title + Summary.
 4. **Save:** Insert into documents table.
 5. **Chunk & Embed:**
 - Split text into 1000-character chunks.
 - Send chunks to OpenAI text-embedding-3-small in batch.
 - Insert vectors into document_chunks .
- **Response:** 200 OK (success message).

B. The Chat Endpoint: POST /api/chat

- **Trigger:** User hits enter in the Dashboard Chat.
- **Payload:** { messages: Message[] } (Standard Vercel AI SDK format).
- **Workflow:**
 1. Get the latest user message.
 2. **Embed:** Send user message to OpenAI to get a query vector.
 3. **Search:** Call match_documents RPC in Supabase with that vector.
 4. **Prompt Engineering:**

```
const systemPrompt = `
You are a helpful Second Brain.
Use the following context to answer the user's question.
If the answer isn't in the context, say "I don't see that in your library
```

CONTEXT:

```
${data.map(d => d.content_chunk).join('\n\n')}
`
```